

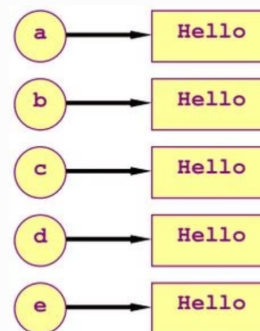


Reference Counting

שיטה לשיתוף ייצוג יחיד של ערך בין מספר אובייקטים — חוסכת זיכרון וזמן, ופותרת את בעיית הבעלות על אובייקטים דינאמיים.

למה בכלל צריך את זה?

הבעיה: כפילות מיותרת



כשמשמשים `a = b = c = d = e =`

"Hello", כל אובייקט מחזיק עותק נפרד של הערך – בזבוז זיכרון וזמן.

הפתרון: שיתוף ערך

כל האובייקטים יצביעו לאותו ייצוג. האובייקט עצמו יודע כמה מצביעים אליו, וכשהמספר יורד ל-0 – הוא מוחק את עצמו.

המצב האידיאלי

שיתוף ערך

כל האובייקטים עם אותו ערך מצביעים לאותו מקום בזיכרון.

מעקב מספור

שדה `refCount` סופר כמה אובייקטים מתייחסים לערך כרגע.

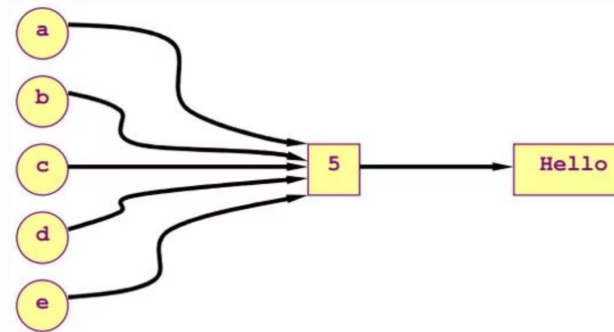
הריסה אוטומטית

כשהמספור יורד ל-0, הערך מוחק את עצמו — ללא צורך בבעלים חיצוני.

מימוש: מבנה StringValue

הרעיון

המספור חייב להיות צמוד לערך, לא לאובייקט. לכן יוצרים מבנה פנימי `StringValue` שמחזיק גם את הנתון וגם את המונה.



refCount

כמה מצביעים לערך זה

char* data

הערך עצמו בזיכרון

בנאים, הורס, והשמה

01

Copy Constructor

מצביע לאותו `StringValue` ומגדיל את `refCount`-1. אין הקצאת זיכרון חדשה!

02

Destructor

מקטין את `refCount` רק אם הגיע ל-0 — מוחק את `StringValue`

03

operator=

מקטין מונה של הערך הישן (ומוחק אם צריך), ומגדיל מונה של הערך החדש.



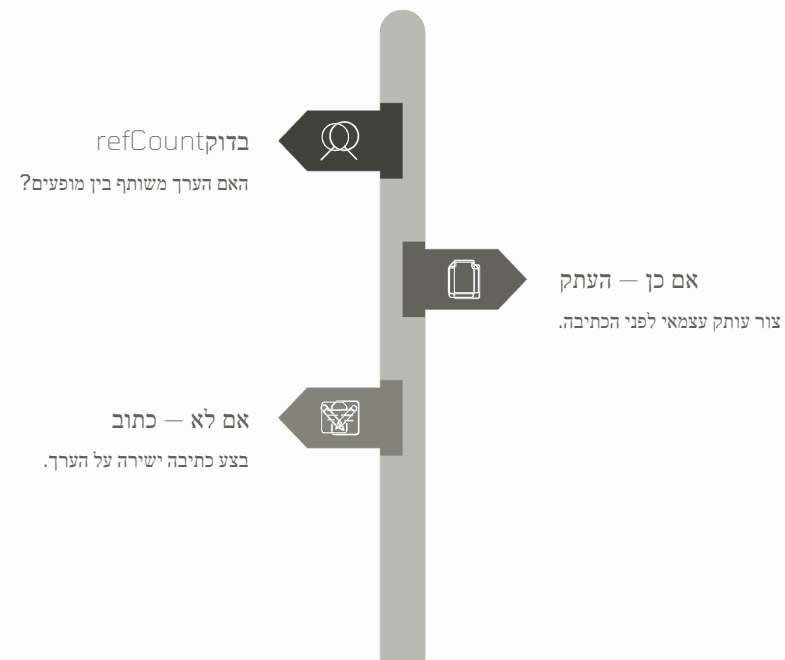
Copy-on-Write

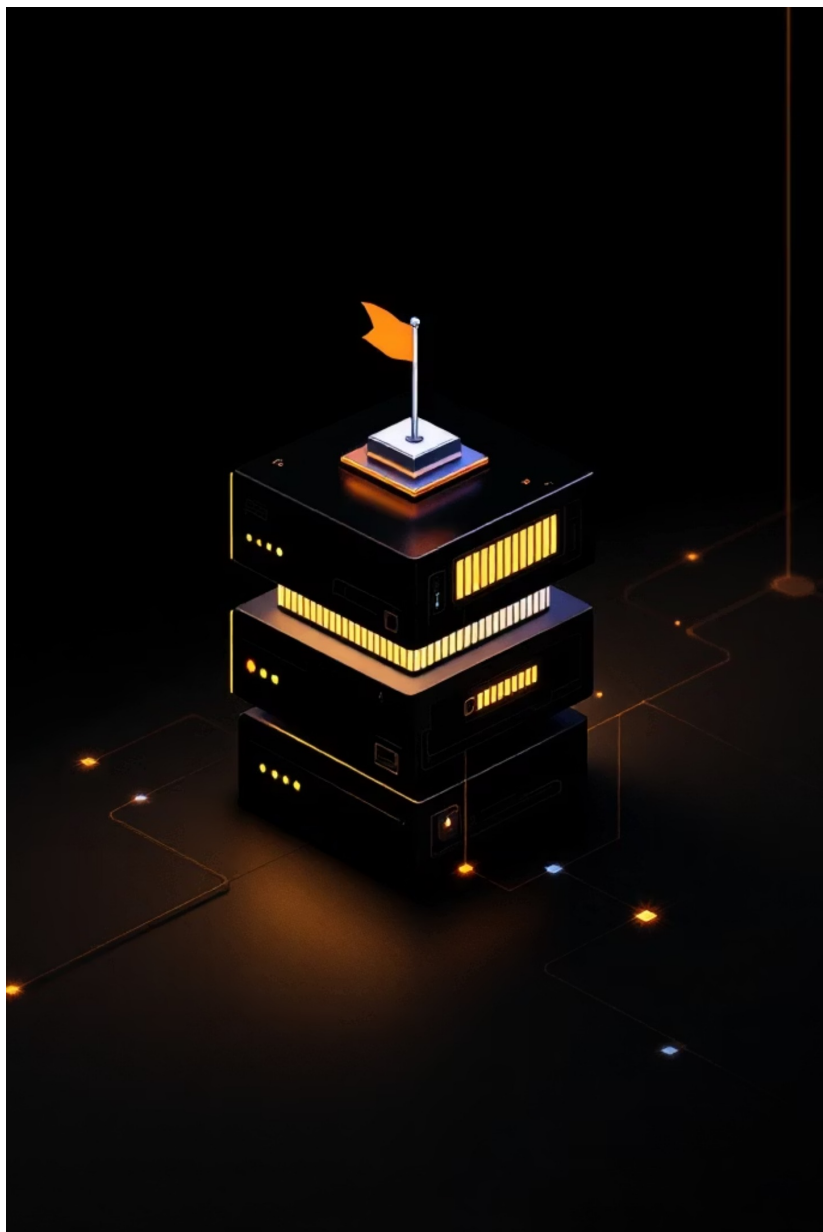
הבעיה

כשמפעילים `operator[]` לא-קבוע, לא ניתן לדעת אם מדובר בקריאה או כתיבה. כתיבה תשנה את הערך לכל האובייקטים המשתפים!

הפתרון

אם יותר מאובייקט אחד מצביע לערך — יוצרים עותק עצמאי לפני הכתיבה. אחרת — כותבים ישירות.





דגל Shareable ומצביעים חיצוניים

כשמחזיקים מצביע חיצוני `char *p = &s[1]` ואז מעתיקים את האובייקט — שני אובייקטים חולקים ערך שניתן לשנות דרך `p`. הפתרון: דגל `shareable`.

`shareable = true`

ברירת מחדל — ניתן לחלוק את הערך

לאחר כתיבה

`shareable = false` — לא ניתן לחלוק יותר לעולם

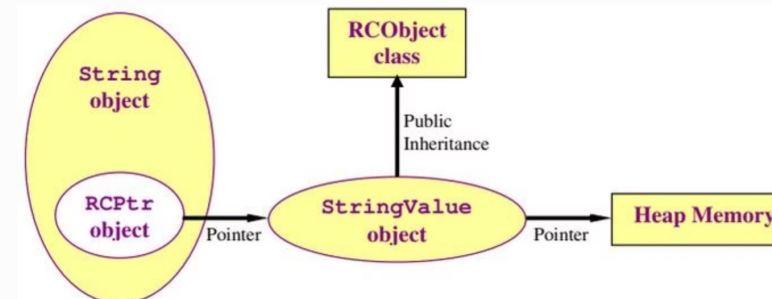
מחלקת בסיס RObject ומצביע חכם RCPtr

מחלקת בסיס RObject

כל מחלקה שרוצה מספור ייחוסים יורשת ממנה. מתחזקת את `refCount` ואת `shareable` ומוחקת את עצמה כש. `refCount == 0`.

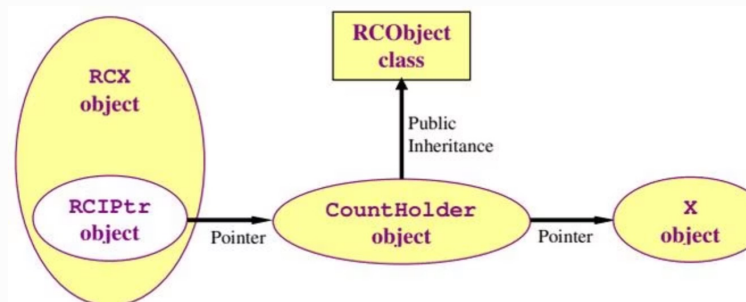
מצביע חכם RCPtr

מטפל אוטומטית בהגדלה/הקטנה של המונה בכל השמה, העתקה והריסה. משחרר את מחלקת `String` מהצורך לממש `Copy Constructor`, `operator=` ו-`Destructor`.



הוספה למחלקות קיימות - RCIPtr

כשאין גישה לקוד המקור של מחלקה X ולא ניתן לגרום לה לרשת מ RCOBJECT-משתמשים ב RCIPtr-עם מחלקת עזר פנימית CountHolder.



→ CountHolder

מחלקה פנימית ב RCIPtr-שורשת מ-
RCObject ומחזיקה מצביע ל-X.

→ RCX

מחלקת עטיפה שמשתמשת ב RCIPtr<X>-
—ללא צורך בשלישיית
Copy/Assign/Destroy.

מתי כדאי להשתמש?

השמות והעתקות רבות

פחות בנייה והריסה של אובייקטים —
חיסכון משמעותי בזמן.

ערכים גדולים

ככל שהערך גדול יותר ורבים חולקים אותו
— חיסכון גדול יותר בזיכרון.

שים לב לעלות

כל ערך סוחב מספור (עלות מקום) ודורש מניפולציות (עלות זמן). אם התנאים לא מתקיימים —
הביצועים עלולים להיפגע.

